

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..9

SERVER EVENTS

 Copy as Markdown

Server events

Serverpod has a built-in event system: post a message to a named channel, and every listener on that channel receives it, even on other servers in a cluster. It is how one user's action reaches everyone else, like the chat-room updates in [streaming's live example](#). You access it through `session.messages`.

Send messages

Send a message with the `postMessage` method. The message is published to the specified channel and must be a [data model](#).

```
var message = UserUpdate(); // Model that represents changes to user data.
await session.messages.postMessage('user_updates', message);
```

In the example above, the message is published on the `user_updates` channel. Any subscriber to this channel in the server will receive the message.

Message scope

Serverpod uses Redis to pass messages between servers. A message is sent to every server connected to the same Redis instance when [Redis is enabled](#), and stays on the current server when it is not. Set the `scope` parameter to choose the delivery explicitly.

Scope	Delivery
<code>MessageScope.auto</code>	Cluster-wide if Redis is enabled, otherwise local. This is the default.
<code>MessageScope.global</code>	Cluster-wide. Throws a <code>StateError</code> if Redis is not enabled.



Ask AI

Scope	Delivery
<code>MessageScope.local</code>	Only to subscribers on the current server.

The default already goes cluster-wide when Redis is enabled. Pass `MessageScope.local` to keep a message on the current server:

```
var message = UserUpdate(); // Model that represents changes to user data.
await session.messages.postMessage(
    'user_updates',
    message,
    scope: MessageScope.local,
);
```

Receive messages

Serverpod provides two ways to handle incoming messages: by creating a stream that subscribes to a channel or by adding a listener to a channel.

Create a stream

To create a stream that subscribes to a channel, use the `createStream` method. The stream emits a value whenever a message is posted to the channel. The typed variant, `createStream<UserUpdate>(...)`, emits an error if a posted message does not match the type.

```
var stream = session.messages.createStream('user_updates');
stream.listen((message) {
    print('Received message: $message');
});
```

In the above example, a stream is created that listens to the `user_updates` channel and processes incoming messages.

Stream lifecycle

The stream is automatically closed when the session is closed. To close the stream manually, cancel the stream subscription.

```
var stream = session.messages.createStream('user_updates');
var subscription = stream.listen((message) {
    print('Received message: $message');
});
```

```
subscription.cancel();
```

Add a listener

To add a listener to a channel, use the `addListener` method. The listener will be called whenever a message is posted to the channel.

```
session.messages.addListener('user_updates', (message) {  
  print('Received message: $message');  
});
```

In the above example, the listener will be called whenever a message is posted to the `user_updates` channel. Listeners receive every message that reaches this server, whatever its scope.

Listener lifecycle

The listener is automatically removed when the session is closed. To manually remove a listener, use the `removeListener` method.

```
var myListenerCallback = (message) {  
  print('Received message: $message');  
};  
// Register the listener  
session.messages.addListener('user_updates', myListenerCallback);  
  
// Remove the listener  
session.messages.removeListener('user_updates', myListenerCallback);
```

In the above example, the listener is first added and then removed from the `user_updates` channel.

Broadcast revoked authentication

The messaging interface also carries authentication revocation events:

`session.messages.authenticationRevoked(userIDentifier, message)` notifies every server that a user's authentication changed, falling back to local delivery when Redis is disabled. For when and how to call it, see [handling revoked authentication](#).